STUDENT MANAGEMENT SYSTEM

- Using C++ and Linked List
- “Simplicity is the soul of efficiency.”

Introduction

- Console-based applicationdevelopedusing C++ and Linked List.
- Manages student records efficiently.
- Stores academic and personal details.

Problem Statement

- Manual maintenance of records
- Data duplication problems
- Difficult searching and updating
- Time-consuming operations

Proposed Solution

- Automated record management
- Dynamic storage using Linked List
- Easy insertion, deletion, and modification

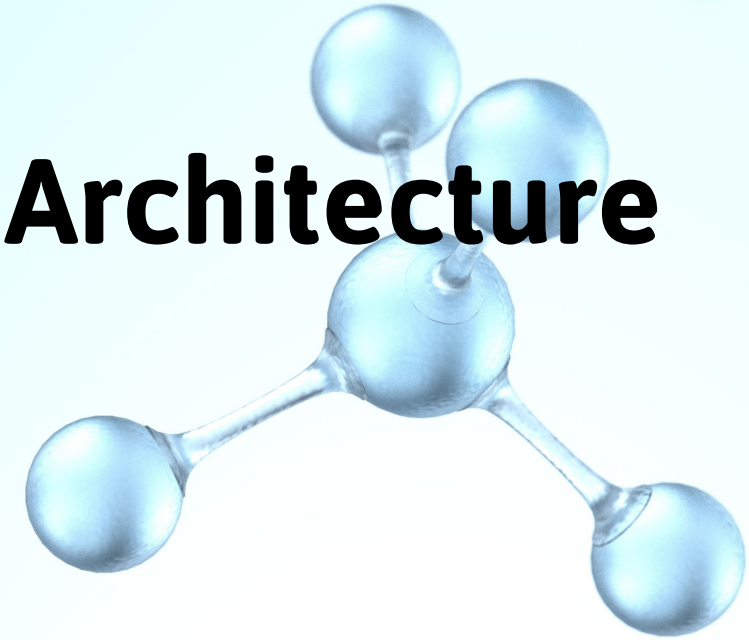
Objectives

- Manage student records efficiently
- Implement Linked List in real application
- Avoid duplicate entries

Technologies Used

- C++
- Singly Linked List
- Classes & Objects
- Pointers

System Architecture



- Input Layer
- Processing Layer
- Data Layer
- User → Functions → Linked List → Output

Linked List Concept

- Dynamic linear data structure
- Nodes connected using pointers
- Efficient insertion and deletion

Node Structure

- Stores Name, Roll Number, Address
- Marks & CGPA
- Pointer to next node

Features of the System

- Add Student
- View Student
- Modify Student
- Delete Student
- Display All

Add Student Operation

- Accept student details
- Check duplicate roll number
- Create and insert node

Search & Modify Operation

- Traverse linked list
- Compare roll numbers
- Update student details

Delete Operation

- Search student node
- Adjust links properly
- Remove node from memory

Display All Students

- Traverse entire linked list
- Display summary of all students

Time Complexity

- Insertion: O(n)
- Searching: O(n)
- Deletion: O(n)

Advantages

- Dynamic memory allocation
- Easy insertion and deletion
- No fixed size limitation

Limitations

- No permanent storage
- Slower for large data
- No graphical interface

Future Enhancements

- Database connectivity
- GUI development
- File handling
- Login authentication

Conclusion

- Efficient student record handling
- Dynamic memory management
- CRUD operations in C++

THANK YOU